Documentation & Reporting

1. User Guide

1.1 Introduction

BlockFit AR is an iOS augmented-reality puzzle game that challenges players to place 3D blocks into matching cut-outs in their real-world environment. Using ARKit's plane detection and intuitive drag-and-rotate controls, players complete levels by accurately aligning pieces within tolerance thresholds.

1.2 Installation Requirements

- iPhone or iPad with **A12 Bionic or newer**
- iOS 15 or above
- Camera permission enabled
- Build through **Xcode** (latest stable version) using a physical device (ARKit not supported on Simulator)

1.3 How to Play

1. Launch the app and allow camera access.
2. Move your device around to detect a horizontal surface.
3. When the AR grid appears, tap **Start** to spawn the puzzle.
4. Drag blocks using touch gestures.
5. Rotate using two-finger rotation.
6. Fit each block into the correct cavity until the piece snaps into place.
7. Complete the level to view your time and star rating.
8. Progress is automatically saved using Core Data.

1.4 Game Progress

- Each level records:
    - Best time
    - Number of stars
    - Completion date

- Players can replay levels to improve their performance.

---

2. Technical Documentation

2.1 System Architecture Overview

BlockFit AR consists of three core layers:

1. **UI Layer (SwiftUI)**
    - Level selection screen
    - Home & About screens
    - Game results and statistics
2. **AR Layer (ARKit + SceneKit)**
    - Horizontal plane detection
    - Rendering of 3D blocks and target cutouts
    - Gesture recognition for drag & rotation
    - Snap-to-target and collision logic
3. **Data Layer (Core Data)**
    - Stores progress per level
    - Provides persistence across sessions
    - Acts as a lightweight game database

2.2 Key Technologies Used

- **ARKit** – plane detection & world tracking
- **SceneKit** – rendering, physics, and 3D objects
- **SwiftUI** – modern UI framework
- **Core Data** – offline progress storage
- **MVVM-inspired project structure** for UI, AR logic, and models

2.3 Core Game Logic

- When the AR session detects a valid plane, a reference anchor is placed.

- The game initializes puzzle objects relative to that anchor.
- Each block tracks its position and rotation in world space.
- If the block enters the tolerance zone of its target position and angle, it safely snaps into place and locks.
- Completion triggers the results screen and updates Core Data.

---

3. Design Decisions

3.1 Why AR?

AR provides a natural, intuitive environment for spatial puzzles. Instead of interacting on a flat 2D screen, players use the real world as part of the game, making the experience more immersive.

3.2 Choice of Frameworks

- **SwiftUI** was chosen for its clean structure, fast iteration, and intuitive state management.
- **SceneKit** is simpler for student-level AR puzzle creation than RealityKit, offering easier collision/physics handling.
- **Core Data** provides lightweight persistence without external storage dependencies.

3.3 Level Design Approach

Levels were created to gradually increase difficulty by modifying:

- Allowed rotation tolerance
- Target shape complexity
- Spatial arrangement of blocks

This ensures a smooth difficulty curve and better usability.

---

4. Development Challenges

4.1 Plane Detection Instability

ARKit sometimes misdetects surfaces under low lighting. This required:

- Adding on-screen instructions
- Using coaching overlays
- Expanding the plane detection threshold for better stability

4.2 Gesture Handling in 3D Space

Translating touch gestures to 3D movement was challenging.
Solutions included:

- Ray-casting from touch positions
- Restricting movement to the detected plane
- Adding damping to avoid jitter

4.3 Snapping Logic

Ensuring snapping feels natural without being too strict required tuning:

- Distance thresholds
- Angle thresholds
- Audio/visual feedback

4.4 Core Data Integration

Ensuring progress stored correctly required careful setup of:

- Managed object context
- Fetch requests
- Error handling

---

5. Testing Results

Comprehensive testing included:

5.1 Unit Testing

- Level model validation
- Persistence logic verification

5.2 Functional Testing

Testers checked:

- Gesture responsiveness
- Plane detection accuracy
- Snapping mechanic
- Level completion flow

5.3 Device Testing

The game was tested on multiple AR-capable devices to ensure:

- Stable tracking
- Smooth rendering
- Proper screen scaling

5.4 User Feedback Summary

Players reported that the game was:

- Easy to understand
- Responsive
- Enjoyable due to the tactile AR interaction

Areas flagged for improvement:

- Lighting sensitivity
- More varied levels
- Additional hints for beginners

---

6. Reflections on the Development Process

The project demonstrated how AR can transform simple puzzles into immersive physical experiences. Developing BlockFit AR provided valuable insight into combining SwiftUI, ARKit, and SceneKit in a cohesive application.

Key reflections:

- **AR requires significant testing in real environments.** Simulators cannot replicate real-world usage.

- **Balancing difficulty is harder in physical puzzles** because the environment affects the experience.
- **SwiftUI + AR bridging requires careful architecture**, but once structured well, it becomes flexible and scalable.
- **Time management mattered greatly** — AR debugging often took longer than expected.
- **Better early playtesting would have helped** refine puzzles sooner and reduce rework.

The project successfully met its objectives: providing a functional, polished AR puzzle game with persistence, intuitive interactions, and clear documentation.